

Class-9 UART Task Code

```
#####
```

```
/*Master Side Code */
```

```
/*
```

```
* File: MasterMain.c
```

```
* Author: sagar
```

```
*
```

```
* Created on 19 May, 2025, 12:03 PM
```

```
*/
```

```
#include <xc.h>
```

```
#include <stdint.h>
```

```
#define _XTAL_FREQ 20000000
```

```
void uartInit(void);
```

```
uint8_t value;
```

```
uint8_t RXvalue;
```

```
void main(void)
```

```
{
```

```
    uartInit();
```

```
    while(1)
```

```
    {
```

```
        value = PORTB;
```

```
        switch(value)
```

```
        {
```

```
            case 0xE0:
```

```
                TXREG = 'A';
```

```
                __delay_ms(100);
```

```
                break;
```

```
            case 0xD0:
```

```
                TXREG = 'B';
```

```
                __delay_ms(100);
```

```
                break;
```

```
            case 0xB0:
```

```
                TXREG = 'C';
```

```
                __delay_ms(100);
```

```
                break;
```

```
            case 0x70:
```

```
                TXREG = 'D';
```

```
                __delay_ms(100);
```

```
                break;
```

```
        }
```

```

if(PIR1 & (0x1U << 5)) // Check RCIF flag
{
    RXvalue = RCREG;
    switch(RXvalue)
    {
        case 'a':
            PORTD = 0x40;
            __delay_ms(100);
            break;
        case 'b':
            PORTD = 0x02;
            __delay_ms(100);
            break;
        case 'c':
            PORTD = 0x42;
            __delay_ms(100);
            break;
        case 'd':
            PORTD = 0x00;
            __delay_ms(100);
            break;
    }
}
}

void uartInit(void)
{

    TRISB = 0xF0;
    PORTB = 0x00;
    TRISD = 0x00;
    PORTD = 0x00;
    OPTION_REG &= ~(0x1U << 7); // Enable Pull Up

    // UART Configuration
    TXSTA = 0x20;
    RCSTA = 0x90;
    SPBRG = 0x09;

    TRISC6 = 0; // TX pin as output
}

```

#####

/* UART Task Slave Side Code */

```
/*
 * File: SlaveMain.c
 * Author: sagar
 *
 * Created on 19 May, 2025, 12:05 PM
 */
#include <xc.h>
#include <stdint.h>

#define _XTAL_FREQ 20000000

void uartInit(void);
void Lcdinit(void);
void LcdCommand(uint8_t i);
void LcdOutput(uint16_t i);
void LcdData(uint8_t i);

uint8_t RXvalue;
uint8_t RxData[9] = {"Rx DATA:"};
uint8_t TxData[9] = {"Tx DATA:"};
void main(void)
{
    uartInit();
    Lcdinit();
    LcdCommand(0x80);
    for(int i = 0; i <= 7; i++)
    {
        LcdData(RxData[i]);
    }

    LcdCommand(0xC0);
    for(int i = 0; i <= 7; i++)
    {
        LcdData(TxData[i]);
    }
    TRISD = 0x00; // PORTD as output
    PORTD = 0x00;

    while(1)
    {
        if(PIR1 & (0x1U << 5)) // Check RCIF flag
        {
            RXvalue = RCREG;
            switch(RXvalue)
            {
                case 'A':
```

```

        LcdCommand(0x88);
        LcdData('A');
        LcdCommand(0xC8);
        LcdData('a');
        TXREG = 'a';
        __delay_ms(100);
        //PORTD = 0x40; // RD6 On, rest Off
        break;
    case 'B':
        LcdCommand(0x88);
        LcdData('B');
        LcdCommand(0xC8);
        LcdData('b');
        TXREG = 'b';
        __delay_ms(100);
        //PORTD = 0x02; // RD1 On, rest Off
        break;
    case 'C':
        LcdCommand(0x88);
        LcdData('C');
        LcdCommand(0xC8);
        LcdData('c');
        TXREG = 'c';
        __delay_ms(100);
        //PORTD = 0x00; // All Off
        break;
    case 'D':
        LcdCommand(0x88);
        LcdData('D');
        LcdCommand(0xC8);
        LcdData('d');
        TXREG = 'd';
        __delay_ms(100);
        //PORTD = 0x42; // RD6 and RD1 On
        break;
    }
}
}
}
}

```

// Function to initialize the LCD

```
void Lcdinit(void)
```

```
{
```

```
    // Set PORTC RC0, RC3 as output (for control signals)
```

```
    TRISC &= ~(0x01 << 0);
```

```
    TRISC &= ~(0x01 << 3);
```

```
    TRISD = 0x00; // Set PORTD as output (for data signals)
```

```
    OPTION_REG &= ~(0x1U << 7); // Enable Pull Up
```

```
    __delay_ms(100); // Wait for LCD to stabilize
```

```
    // LCD initialization sequence as per HD44780 LCD datasheet
```

```

    LcdCommand(0x30); // Send function set command (8-bit mode)
    __delay_ms(100);
    LcdCommand(0x30); // Repeat function set command
    __delay_ms(100);
    LcdCommand(0x30); // Repeat function set command again
    __delay_ms(100);
    LcdCommand(0x38); // Set LCD for 8-bit mode, 2-line display, 5x8 font
    __delay_ms(100);
    LcdCommand(0x0C); // Turn on display, cursor off
    __delay_ms(100);
    LcdCommand(0x01); // Clear the display
    __delay_ms(100);
    LcdCommand(0x06);
    __delay_ms(100);
}

```

// Function to send data (characters) to the LCD

```

void LcdData(uint8_t i)
{
    PORTC |= (0x1 << 3); // Set RS (RC3) = 1 (indicates data mode)
    PORTD = i;           // Place data on PORTD
    PORTC |= (0x1 << 0); // Set EN (RC0) = 1 (enable pulse start)
    __delay_ms(100);     // Small delay for command execution
    PORTC &= ~(0x1 << 0); // Set EN (RC0) = 0 (enable pulse end)
}

```

// Function to send commands to the LCD

```

void LcdCommand(uint8_t i)
{
    PORTC &= ~(0x1 << 3); // Set RS (RC3) = 0 (indicates command mode)
    PORTD = i;           // Place command on PORTD
    PORTC |= (0x1 << 0); // Set EN (RC0) = 1 (enable pulse start)
    __delay_ms(100);     // Small delay for command execution
    PORTC &= ~(0x1 << 0); // Set EN (RC0) = 0 (enable pulse end)
}

```

void uartInit(void)

```

{
    TRISB = 0xF0;
    PORTB = 0x00;
    OPTION_REG &= ~(0x01 << 7);

    // UART Configuration
    TXSTA = 0x20;
    RCSTA = 0x90;
    SPBRG = 0x09;

    TRISC7 = 1; // RX as input
}

```